Parallel Web Crawler

Presentation & Discussion Guide

Team Division · What to Say · Expected Questions · Code Modifications

Team: [Ragnarok] | Course: CSE373 | 2025-2026

1. Team Division — Who Owns What

Member	Role	Files Owned	Key Topics to Know
Member 1	Core Engine	CrawlerEngine.java	Thread pool, BlockingQueue, BFS, AtomicInteger, retry logic, stage timing, BenchmarkResult
Member 2	GUI	MainController.java CrawlerApp.java MainView.fxml dark-theme.css	Platform.runLater(), AnimationTimer, benchmark mode flow, speedup calculation, onStart/onStop/onClear
Member 3	Storage	CsvStorage.java JsonStorage.java SqliteStorage.java	Why synchronized, NDJSON format, SQLite schema (pages + links tables), batch inserts, transactions
Member 4	Benchmarking	BenchmarkResult ConsoleDisplay.java Performance analysis	Speedup formula S=T_seq/T_par, Amdahl's Law, efficiency, bottlenecks, which stage benefits most
Member 5	Architecture & Docs	CrawlerConfig.java CrawledPage.java module-info.java README.md All diagrams	Builder pattern, module system, BFS algorithm, all 6 UML diagrams, project structure, how to run

2. What to Say — Presentation Script

Opening (30 seconds)

"Our project is a Parallel Web Crawler. A web crawler visits web pages automatically and collects information — the same technology Google uses to index the internet. Our version does this using **parallel threads** to visit multiple pages simultaneously, which is the core parallel programming concept we're demonstrating."

Explaining the Parallel Concept (1 minute)

"Imagine you need to visit 100 web pages. If you visit them one by one — sequential — and each takes 1 second, total time = 100 seconds. With 6 threads running in parallel, you visit 6 pages at the same time. Total time ≈ 100/6 ≈ 17 seconds. That's a **6x speedup**."

"The challenge is making this thread-safe. When 6 threads all want to update the same counter or check the same URL set, we need to handle that carefully. We use ConcurrentHashMap, AtomicInteger, and synchronized methods to avoid race conditions."

Explaining the Pipeline (1 minute)

"Our system works in 5 stages: (1) User enters a URL. (2) The crawler discovers links on each page. (3) A thread pool fetches multiple pages in parallel. (4) Each thread extracts data from its page. (5) Results are saved to CSV, JSON, and SQLite simultaneously. Parallelism is applied in stages 2, 3, 4, and 5."

Showing the Demo

"In the GUI, you can see: the live log on the left showing each crawled page with its HTTP status and response time. The table on the right shows all pages found. The stats at the top show total pages, success count, errors, and links found. With Benchmark Mode enabled, you can see the speedup between sequential and parallel directly."

3. Expected Discussion Questions & Answers

Q1: Why a Web Crawler and not the suggested Image Pipeline?

A: The course policy explicitly allows original ideas. A web crawler satisfies all four core requirements: clear parallelism (thread pool), multi-stage pipeline (discover→fetch→extract→store), proper synchronization (ConcurrentHashMap, AtomicInteger), and performance comparison (sequential vs parallel modes). It also reflects real-world systems — Google's crawler, Bing, and SEO tools all work this way.

Q2: Why use a thread pool instead of creating a new thread per URL?

A: Creating a thread has overhead: it allocates stack memory (typically 512KB–1MB), registers with the OS scheduler, and must be garbage collected when done. With 1000 URLs, creating 1000 threads would consume 1GB of memory just for stacks. A fixed pool of 6 threads reuses 6 workers for all 1000 URLs, keeps memory constant, and allows us to control the degree of parallelism.

Q3: How do you prevent two threads from crawling the same URL?

A: We use ConcurrentHashMap.newKeySet(). The critical operation is visitedUrls.add(href). This is atomic — it checks if the URL exists AND inserts it in one indivisible CPU step. If threads A and B simultaneously call add() with the same URL: one gets true (new — add to queue), one gets false (already there — skip). With a regular HashSet, both could see 'not visited', both add to queue, both crawl the same URL — a race condition.

Q4: Why is pageCount an AtomicInteger and not a regular int?

A: A regular int count++ is actually three operations: read, add 1, write. With two threads: both read 5, both add 1, both write 6. We lose one increment. AtomicInteger.incrementAndGet() uses a hardware CAS (Compare-And-Swap) instruction that does all three atomically. This is important for the maxPages check — if pageCount is inaccurate, we might crawl too many or too few pages.

Q5: What happens when a page fails to load?

A: The crawlPageWithRetry() method catches IOException. On first failure (attempt=0), it calls itself recursively with attempt=1 (retry once). If that also fails, it returns a CrawledPage with statusCode=0 and the error message in the description field. This is stored in all output files so we can see which URLs failed and why. The GUI shows failed pages in red in the status column.

Q6: How is the GUI updated from background threads?

A: JavaFX has a rule: only the Application Thread can touch UI components. Our worker threads run on background threads. We use Platform.runLater(Runnable) to schedule GUI updates. JavaFX queues these Runnables and runs them on the Application Thread between frames. This keeps the GUI responsive at 60fps while the crawler runs in the background.

Q7: How do you measure speedup?

A: With Benchmark Mode, the system runs the same crawl twice. First with threadCount=1 (triggers runSequential() — no thread pool, one URL at a time). We record T_sequential. Then with the configured thread count (triggers runParallel() — full thread pool). We record T_parallel. Speedup =

T_seq / T_par. Efficiency = Speedup / N × 100%. Expected results with 6 threads on a web crawl: 3–5x speedup, 50–83% efficiency.

Q8: Why is the speedup less than N for N threads?

A: This is explained by Amdahl's Law. Not all parts of the program can be parallelized. Some parts are inherently sequential: the main loop, the queue draining, the final storage finalization. Also, threads compete for ConcurrentHashMap access, network bandwidth, and CPU. The synchronized storage writers also create some serialization. These sequential and contended portions limit the maximum achievable speedup.

Q9: What design patterns do you use?

A: We use three:

- **Builder Pattern:** CrawlerConfig.Builder — constructs complex config objects step by step
- **Producer-Consumer:** Main loop produces URL tasks; thread pool workers consume them
- **Observer/Callback:** onPageCrawled callback lets MainController react to each crawled page

Q10: What would you improve with more time?

A: Several improvements are possible:

- robots.txt support — respect crawl rules that websites publish
- Rate limiting per domain — avoid hammering one server with too many threads
- Distributed crawling — spread across multiple machines for very large crawls
- Graph visualization — display the link graph from the SQLite links table
- Content analysis — detect language, extract keywords, classify page types

4. Code Modifications Made (Phase 2+3)

The following modifications were made to the original project code. Each is explained with: what changed, why we changed it, and the complete updated class.

4.1 CrawlerEngine.java — Complete Updated Class

Changes from original:

- **Added runSequential():** New method for sequential benchmark mode. Triggered when threadCount==1.
- **Added runParallel():** Extracted from the original start() method and enhanced with per-stage timing.
- **Renamed crawlPage() to crawlPageWithRetry():** Added retry logic — catches IOException and retries once.
- **Added BenchmarkResult record:** Immutable data object holding per-stage timing (discoveryMs, fetchMs, extractionMs, storageMs).
- **Added stage timing variables:** stageDiscoveryMs, stageFetchMs, stageExtractionMs, stageStorageMs — accumulated per crawl.
- **Added onStageComplete callback:** Lets MainController display timing message in the log when crawl finishes.
- **Added saveAndNotify() method:** Extracted repeated storage+callback code into one reusable method.

Why: The original code had no benchmarking ability and no retry logic. These are mandatory for Phase 2 (retry) and Phase 3 (performance comparison).

4.2 MainController.java — Complete Updated Class

Changes from original:

- **Added benchmark mode logic in onStart():** If benchmarkModeCheck is selected, runs sequential crawl first then parallel.
- **Added runCrawl() method:** Extracted crawl setup from onStart() into a reusable method that can be called twice for benchmarking.
- **Added displaySpeedupResult():** Calculates and displays Speedup + Efficiency in the GUI log.
- **Added updateStageLabels():** Updates the four stage-timing labels from a BenchmarkResult object.
- **Added seqTotalMs / parTotalMs fields:** Store timing from sequential and parallel runs for comparison.
- **Added isRunningSeqBenchmark flag:** Prevents storage writes during the sequential benchmark run (we only want results from the parallel run).

Why: The original controller had no benchmark mode. The comparison between sequential and parallel is mandatory for the Phase 3 report.

4.3 No Changes to Other Files

The following files are **unchanged** from the original project — they were already correct and complete:

- CrawlerConfig.java — Builder pattern was already clean

- CrawledPage.java — Data model was complete
- CsvStorage.java — Synchronized writer was correct
- JsonStorage.java — NDJSON format + finalize() was complete
- SqliteStorage.java — Pages + links schema with batch insert was complete
- CrawlerApp.java — JavaFX launcher was correct
- ConsoleDisplay.java — Terminal output utility was complete
- Main.java — Entry point was correct (may need minor update for new BenchmarkResult)

Summary: Only 2 files were modified (CrawlerEngine.java and MainController.java). All other 8 files are used as-is from the original implementation.